Q1 - 3 Marks

Explanation - `102 = 01100110` in 8 bits, Taking two's complement : `01100110 → 10011001` (flip all bits), `10011001 + 1 = 10011010`

[1] for writing `102 = 01100110`

[3] for all 8 correct bits - `10011010`

[0] for anything else

Q2 - 3 Marks

```
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        printf("%d ", i - j);
    }
    printf("\n");
}
```

Explanation - `i` goes from `0` to `2`, and for every value of `i`, `j` also goes from `0` to `2`.

`0 -1 -2` for i = 0,

`1 0 -1` for i = 1

`2 1 0` for i = 2

[+1] for each correct line of output

[-1] if required new lines are omitted

[0] for wrong output

Q3 - 3 Marks

```
int a = 0, b = 0;

while(a++ < 3);
b++;

printf("%d %d", a, b);
```

Explanation - Notice the semicolon after the `while` loop. The loop has an empty body. When a is incremented to 3, `3 < 3` → false, but `a++` still increments `a` to `4` . So correct output is `4 1`

[1] for correct final value of `a = 4`

[+1] for correct final value of `b = 1`

[3] for complete correct output `4 1`

[0] for anything else

Q4 - 3 Marks

```
int i = 2026;

for (; i; i >>= 1)
    printf("Cpro\n");
```

Explanation - The loop continues while `i != 0`. Every iteration, `i` is right-shifted by one bit. `2026 in binary = 11111101010` . It takes `11` right shifts for the value to become `0`. Therefore, `Cpro` is printed `11` times in new lines.

[1] for `2026 in binary = 11111101010`

[3] for writing the complete correct output, with `Cpro` on `11` separate lines

[-1] if newline is omitted

[0] for anything else

Q5 - 3 Marks

```
int a = 2, b = 4, c = 8;
int result = a + b * c / a % b - --c;
printf("result = %d, c = %d \n " , result, c);
```

Explanation -

b * c = 4 * 8 = 32
32 / a = 32 / 2 = 16
16 % b = 16 % 4 = 0
--c = 7

result = 2 + 0 - 7
= -5

Correct output - `result = -5, c = 7` OR `result = -3, c = 7`

[1] for correctly getting `c = 7`

[+1] for correctly getting `result = -5` OR `result = -3`

[3] for complete correct output (writing “result” and “c” in the output)

[0] for anything else

Q6 - 3 Marks

Sign: 0
Exponent: 10000010
Mantissa: 10001000000000000000000

Explanation - `Sign 0 → Positive`, `Exponent 10000010₂ = 130`, `Actual exponent = 130 - 127 = 3` ,

Mantissa 1.10001_2
= 1 + 1/2 + 1/32
= 1.53125

Multiply the exponent
1.53125×2^3
= 1.53125 × 8
= 12.25

[1] for correctly calculating exponent as `3`

[3] for final answer `12.25` or `1.53125 × 2³`

[0] for anything else

Q7 - 3 Marks

```
int x = 150;
int enable = 0;

if (0 < x < 100 || enable)
    printf("Condition met!\n");
else
    printf("Condition failed!\n");
```

Explanation - The condition is evaluated in the following order - `(0 < x) < 100` → `0 < 150` → `1` → `1 < 100` → `1` → Condition is true

[1] for identifying that `0 < x` evaluates to `1`

[3] for complete correct output `Condition met!`

[0] for anything else

Q8 - 3 Marks

```
int sum = 0;
int i = 1;

while (i <= 10) {
    if (sum + i > 12) {
        break;
    }

    sum += i;
    i += 2;
}
```

Explanation -

i	sum + i	Action	New sum
1	1	Add	1
3	4	Add	4
5	9	Add	9
7	16	Break	9

Hence the correct output is `sum = 9, i = 7`

[1] for correct final `sum = 9`

[+1] for correct final `i = 7`

[+1] for complete correct output (writing “sum” and “i” in the output)

[0] for anything else

Q9 - 3 Marks

```
float x = sqrt(2.0);

if (x * x == 2.0)
    printf("Perfect square!\n");
else
    printf("Not exact\n");
```

Explanation - `sqrt(2.0)` cannot be represented exactly as a finite binary floating-point number. After storing the result in a `float`, rounding occurs, so squaring the stored value does not give exactly `2.0`. Hence the condition evaluates to false.

[1] for correct output `Not exact`

[+2] for mentioning floating-point precision/rounding and for explaining that `x * x` is not exactly equal to `2.0`

[0] for anything else

Q10 - 3 Marks

```
short x = 1;

while (x >= 0) {
    x++;
}

printf("%hd\n", x);
```

Explanation - **** keeps incrementing until it reaches `32767 i.e 2^15 - 1`. The next increment produces `-32768` . Now `x >= 0`is false, and the loop terminates.

[1] for identifying maximum positive value as `32767`

[3] for final answer `-32768`

[0] for anything else

Q11 - 3 Marks

```
int n = 13;

for (int i = 1; i <= 4; i++) {
    if (n & 1)
        n = n >> 1;
    else
        n = n << 1;

    printf("%d ", n);
}
```

Explanation -

Initially `n = 13`

Iteration 1

13 & 1 = 1
n = 13 >> 1 = 6

Iteration 2

6 & 1 = 0
n = 6 << 1 = 12

Iteration 3

12 & 1 = 0
n = 12 << 1 = 24

Iteration 4

24 & 1 = 0
n = 24 << 1 = 48

Correct output: `6 12 24 48`

[+0.5] for each correct value in sequence

[3] for complete correct output `6 12 24 48`

[0] for anything else

Q12 - 3 Marks

```
int i,j;
for (i = ___; i <= ___; i++) {
    for (j = ___; j <= ___; j++) {
        printf("%d ", ___);
    }
    printf("\n");
}
```

Correct fill in the blanks -

```
for (i = 1; i <= 8; i++) {
    for (j = 1; j <= 10; j++) {
        printf("%d ", i * j);
    }
    printf("\n");
```

```
}
```

[1] for correctly completing the outer loop (`i = 1`, `i <= 8`)

[1] for correctly completing the inner loop (`j = 1`, `j <= 10`)

[1] for correctly writing the expression `i * j`

[0] for anything else

Q13 - 4 Marks

```
int n;

int prev = n & 1;
n >>= 1;

while (n > 0) {
    int curr = n & 1;

    if (curr == prev) {
        printf("Not alternating");
        return 0;
    }

    prev = curr;
    n >>= 1;
}

printf("Alternating");
```

[1] Correct Logic for extracting/checking the current bit such as `n & 1`

[+1] for correctly comparing consecutive bits and updating cur and prev

[+1] for correctly updating/shifting to the next bit

[+1] for correct overall logic and output/result

[0] for anything else